

Final Project: Mini-Movie Making with dynamic 3DGS

Haeun Jeon (2023100086)

Dept. Computer Science and Engineering, Korea University

Introduction to Computer Vision (DATA302)

Prof. Gyeongsik Moon

Dec. 7, 2025

1. Data Collection

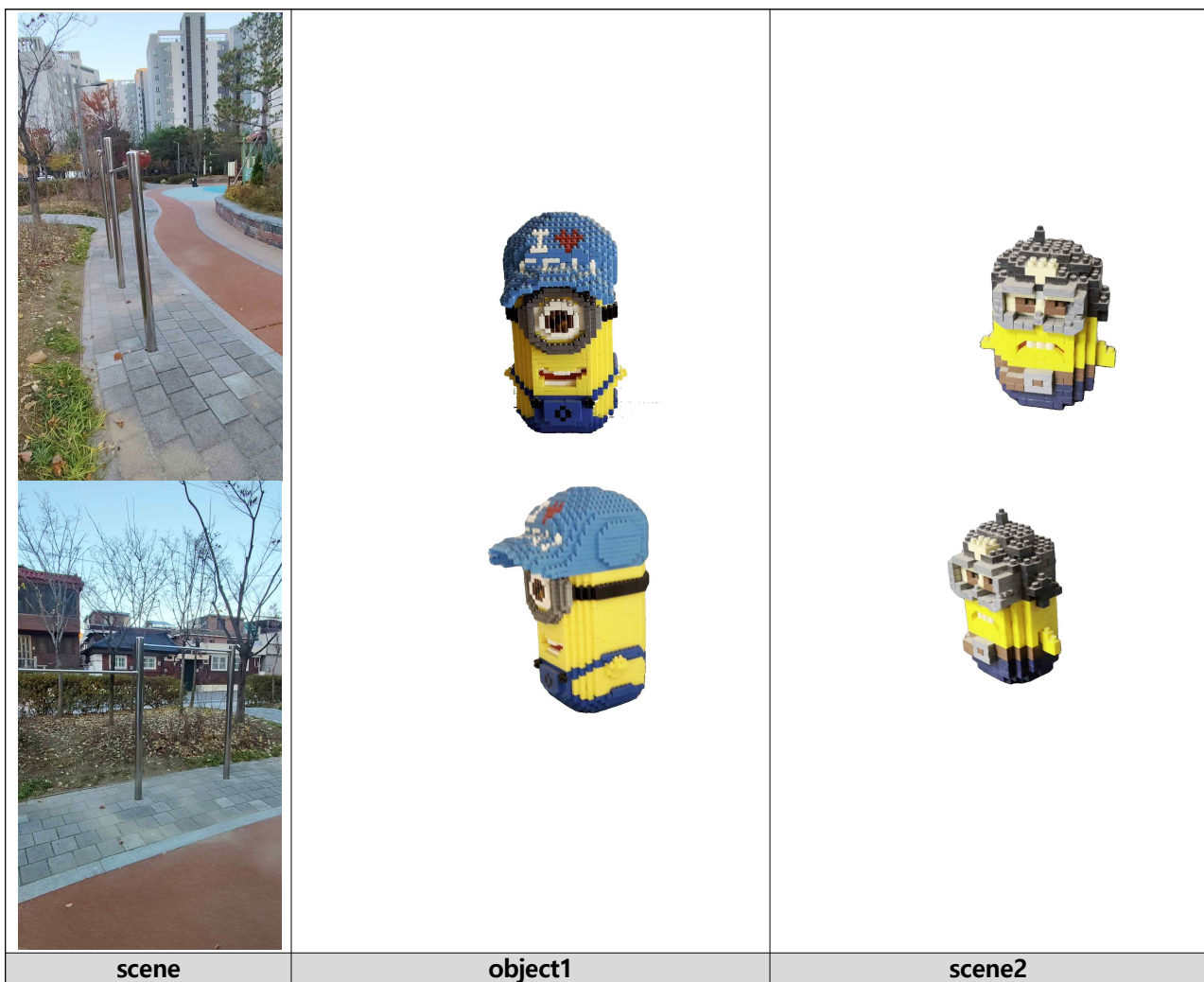
1.1 scene video

The scene was recorded at a playground near my house. COLMAP extracted frames and features at 30 FPS and performed Structure-from-Motion (SfM). As a result, a total of 771 frames were generated.

1.2 object video

The object is a Minions nanoblock figure, and it was recorded separately. COLMAP extracted frames and features at 7.5 FPS and performed SfM, which resulted in 177 frames for object1 and 128 frames for object2.

Initially, when the background was not removed, the floor and wall were also reconstructed together, and even with extensive filtering the 3DGS reconstruction was noticeably degraded. Therefore, SAM2 was used to segment the object and extract only the foreground region, and SfM was applied again on the segmented object-only frames.



2. 3DGS Reconstruction

I used the [official](#) implementation of 3D Gaussian Splatting. For the scene video, the number of iterations was set to 12,000, because the FPS of the extracted frames was very high, resulting in extremely long training time. For the object videos, the iterations were set to 30,000. After completing 3DGS reconstruction, I rendered the results using the official rendering script

(gaussian-splatting/render.py), and the output results are as follows.

qualitative result	- iter 7000 ◦ loss : 0.0376 ◦ PSNR : 22.8915 - iter 12000 ◦ loss : 0.0322 ◦ PSNR : 23.1485	- iter 7000 ◦ loss : 0.0054 ◦ PSNR : 31.3458 - iter 30000 ◦ loss : 0.0032 ◦ PSNR : 35.3826	- iter 7000 ◦ loss : 0.0058 ◦ PSNR : 30.9239 - iter 30000 ◦ loss : 0.0045 ◦ PSNR : 34.4582
quantitative result			
	scene	object1	object2

3. Scene-object Composition

(The following scripts—compose.py, render_composed.py, and animate_final_composition.py—must be executed inside the cloned official 3DGS repository by copying them into the top-level gaussian-splatting folder and running them from there.)

3.1 Composition

Command : python compose.py --scene ../output/scene_3_final \
--objects ../output/minion_2_gaussian ../output/minion_gaussian \
--output ../output/composed_minions \
--remove_pattern

The composition module is implemented in compose.py. The ComposedGaussianModel class enables merging multiple Gaussian models into a single unified representation. When adding each object using the add_model method, spatial transformations are applied based on a predefined list of transformations:

- **Scale:** logarithmic scaling applied to position coordinates and scaling parameters
- **Rotation:** Euler angles → rotation matrix → quaternion-based rotation applied to coordinates
- **Translation:** rigid transformation followed by 3D translation

During initial trials, SAM2-based background removal resulted in white background artifacts that dominated the rendered images after composition. Therefore, a filtering step was added to suppress background pixels with white color patterns.

Finally, all Gaussian parameters (xyz, features, scaling, rotation, opacity) were concatenated

into numpy arrays to form the composed scene representation.

3.2 Composition Rendering

Command : `python render_composed.py --scene ../output/scene_3_final --composed_ply ../output/composed_minions/composed.ply --output ../output/composed_minions`

`render_composed.py` renders the composed result. Camera parameters are loaded from the original scene model, while the composition script exports the merged Gaussian representation as a `.ply` file. The renderer then loads Gaussian parameters from the composed `PLY` and calls `gaussian_render.render()` for each camera viewpoint, saving the rendered outputs as PNG frames.

3.3 Composition and Rendering Results



4. Dynamic Rendering

4.1 Intended Scenario

- **Theme:** Minions' Outdoor Adventure
- **Narrative structure:**
 - Opening scene → object 1 appears → camera movement within the scene → object 2 appears → dynamic camera motion across the playground
 - Object behavior : each object appears at a start point and settles at an end point
- **Implementation concept** : generate a JSON animation schedule and render frames based on the JSON

4.2 Implementation

Command : `python animate_final_composition.py --scene ../output/scene_3_final --object1 ../output/minion_2_gaussian --object2 ../output/minion_gaussian --output ../output/final_animation --json animation_config.json`

The animation pipeline consists of two stages:

- **Stage 1 - JSON Generation** (def generate_animate_json)

- A JSON animation schedule is generated that defines, for each frame, the camera index and the transformation applied to each object. The animation is divided into phases:

Frame range	Behavior
0-30	scene only, camera 200 → 230
31-90	object 2 appears and moves
91-160	camera transition (230 → 300)
161-220	object 1 appears and moves
221-280	object scale change (sinusoidal)
281+	depth-aware composition, camera 300 → 605, objects remain fixed

- To determine the object transformations, the start and end target positions were iteratively tuned by adjusting translation, rotation, and scale values in compose.py. The video opens not from camera index 0 but from camera index 200 so that the bar structure in the playground is centered in the first shot. While tuning transformations, I discovered the following key relationships:

- Moving an object left/right relative to camera index 0 requires updating the z-translation (positive: closer to the left side / negative: toward the right side)
- To enlarge and push an object forward, translation in the x-axis should increase gradually

- **Stage 2 - Rendering from JSON** (def render_from_json)

- The JSON file is parsed to load per-frame parameters
- For each frame:
 1. a ComposedGaussianModel is created
 2. the scene Gaussians are added
 3. object Gaussians are added with transformations from the JSON
 4. the composed model is transferred to GPU for rendering
 5. the frame is rendered with the corresponding camera and saved as PNG

- After all PNG frames are generated, the final video is constructed using FFmpeg

(Commend : ffmpeg -framerate 30 -i ../output/final_animation/frame_%04d.png -c:v libx264 -pix_fmt yuv420p ../output/final_animation/animation.mp4)

4.3 Dynamic sequence snapshots

